Software Requirements Specification

for

Campus Hub

Version 1.0 approved

Prepared by

Hoor Spogmay
Aqsa Ahmed
Seemab Malik

Riphah International University

6-10-2024

Table of Contents

Name	**Sap id**	**Version**
Aqsa Ahmed	53106	Version 1 (Approved)
Seemab Malik	54910	Version 2 (Approved)
Hoor Spogmay	53541	Version 3 (Approved)

1. Introduction

1.1 Purpose

The purpose of this document is to describe the software requirements for the software **campus hub**. It is an Education management System (EMS). This system will provide an all-in-one platform for educational institutions to manage student records, track attendance, handle fee payments, and store data efficiently.

1.2 Scope

Campus hub will streamline administrative processes, reduce human errors, and improve decision making by centralizing student data. It will support administrators, teachers and students in managing data related to courses, fees, attendance, and personal information. This app will be developed using Java technology.

1.3 Definitions, Acronyms and abbreviations

- **EMS:** Education Management System
- **CRUD:** Create, Read, Update, Delete
- **UI:** User Interface
- **SRS:** Software Requirement Specifications

1.4 References

This SRS document is based on the project proposal submitted (**Campus Hub**)

2. Overall Description

2.1 Product Perspective

Campus Hub will serve as a centralized system that integrates with an institution's existing infrastructure. It will offer features to manage student data,

course information, attendance tracking, and fee management. It provides roles-based access to administrators, students and teachers

2.2 Product Features

- **Centralized Student Data Management:** Securely store and manage academic and personal information.
- **Attendance Tracking:** Record and report student attendance.
- **Fee Management:** Manage and track student fee payments.
- **Course Management:** Allocate and manage courses for students and teachers.
- **Instructor Dashboard:** Manage assignments, make announcements, and monitor student progress.

2.3 User Classes and Characteristics

- **Administrators**
- **Teachers**
- **Students**

2.4 Operating Environment

Campus Hub will be a desktop based app that will be accessible on any type of desktop computers or laptops.

2.5 Design and Implementation Constraints

- The system will be developed using **Java** technology.
- The system must comply with educational data protection regulations and ensure role-based access.

3. System Features

3.1 Admin Capabilities

1. Course Assignment: Admins can assign courses to educators and students, including adjusting course details as necessary

2. Financial Oversight: Admins manage all financial transactions related to student fees, including tracking payments and overdue balances with automated notifications.
3. Schedule Coordination: Admins create, modify, and view class schedules for faculty and students, ensuring optimal resource allocation.
4. Record Maintenance: Admins can update and maintain student academic and personal records, including attendance data, with a secure audit trail.

3.2 Teacher Capabilities

1. Notification System: Teachers can post announcements visible to their students and set reminders for important deadlines.
2. Course Oversight: Teachers manage their courses by uploading materials, setting assignments, and grading submissions, with the ability to track student progress.
3. Access to Student Information: Teachers can review individual student records, including academic performance, attendance, and engagement metrics.

3.3 User Capabilities

1. Personal Record Management: Students can manage and update their academic and personal information.
2. Attendance Review: Students can access and track their attendance records and receive alerts for any discrepancies.
3. Fee Monitoring: Students can check their payment history, outstanding fees, and receive notifications for upcoming payments.
4. Contact Information Management: Students can update their personal and guardian contact details.
5. Course Registration: Students can enroll in courses, view related materials, and manage their schedules.

4. . External Interface Requirements

4.1 User Interfaces

- A clean, intuitive user interface designed for non-technical users.
- Different dashboards for admins, teachers, and students based on role-based access

- Interactive views for student records, attendance, and fee management.

4.2 Software interfaces

- Integration with payment gateways for fee processing.
- APIs for future integration with other educational systems.

5. Non-Functional Requirements

5.1 Performance

- The system must support at least 50 concurrent users without significant performance degradation.
- The system response time should not exceed 2 seconds for most operations

5.2 Security

- All sensitive data (student records, fee information) must be encrypted.
- Role-based access control must be implemented to ensure data confidentiality.

5.3 Usability

- The system must be easy to use with minimal training, especially for non-technical users like students and teachers.

5.4 Reliability

- The system must have 99.9% uptime.
- Automated backups must occur daily to prevent data loss.4.

5.5 Scalability

- The system must be scalable to accommodate more users and data as the institution grows.

6. Use Cases

6.1 Student Record Management

- **Actors:** Admin, Teachers, Students
- **Precondition:** The user must be logged in with the correct permissions.
- **Main Flow:**
  - The Admin logs into the system and goes to the "Student Records" section.
  - The Admin can add, update, or remove student information (personal details, academic history).
  - Teachers can access records of students in their classes.
  - Students can view their personal records.
- **Postcondition:** The student information is updated in the database.

6.2 Course Allocation Management

- **Actors:** Admin, Teachers, Students
- **Precondition:** The user must be logged in with the appropriate role.
- **Main Flow:**
  - The Admin logs in and navigates to the "Course Allocation" section.
  - The Admin assigns courses to students and teachers.
  - Teachers manage the courses they are assigned to.
  - Students can browse available courses and enroll.
- **Postcondition:** Course assignments are successfully recorded and visible to the respective users.

6.3 Financial Management

- **Actors**: Admin, Students
- **Precondition**: The user must be authenticated with the proper permissions.
- **Main Flow:**
  - The admin logs in and accesses the "Fee Management" section.
  - The admin updates the fee structure, logs payments, and monitors unpaid fees.
  - Students can check their payment history and see any outstanding fees.

- **Postcondition:** The financial status is accurately updated in the system.

6.4 Attendance Management

- **Actors:** Admin, Teachers, Students
- **Precondition:** The user must be authenticated with the correct role.
- **Main Flow:**
  - Attendance is recorded in the system by either Teachers or the Admin.
  - Students can view their individual attendance records.
  - Admins can generate comprehensive attendance reports.
- **Postcondition:** Attendance data is saved and accessible to relevant parties.

6.5 Announcement Management

- **Actors:** Teachers, Students
- **Precondition:** The user must be logged in with the necessary role.
- **Main Flow:**
  - A Teacher creates an announcement and submits it.
  - The announcement is displayed for all students assigned to that class.
  - Students can view the announcement in their dashboards.
- **Postcondition:** The announcement is successfully displayed on the students' dashboards.

7. Backlog

7.1 Product Backlog

The **Product Backlog** is a prioritized list of all features, enhancements, and tasks that need to be developed over the course of the project. It includes high-level user stories and epics that describe the system's required functionalities.

The product backlog for **Campus Hub** may include the following **user stories** (examples):

1. **Manage Student Records**
   - As an admin, I want to add, edit, and delete student records so that I can keep the database up to date.
   - *Priority*: High
2. **Track Attendanc**

- As a teacher, I want to mark attendance so that I can keep track of student presence.
- *Priority*: High

3. **Fee Management**
    - As a student, I want to view my fee payment status so that I can manage my payments.
    - *Priority*: Medium
4. **Instructor Dashboard**
    - As a teacher, I want a dashboard to manage my courses, post assignments, and send announcements to students.
    - *Priority*: Medium
5. **Course Management**
    - As an admin, I want to allocate courses to students and teachers so that they can access course materials.
    - *Priority*: High

The product backlog will evolve over time, with new items added or reprioritized based on feedback from stakeholders and the progress of the project.

7.2 Sprint Backlog

The **Sprint Backlog** is a subset of the product backlog that is selected for completion during a specific sprint. It is typically chosen during the sprint planning meeting and contains only the user stories or tasks that the team will work on during that sprint.

Example of a sprint backlog for **Sprint 1**:

- **Create Student Record CRUD Functionality**
    - Implement a form for admins to input new student records.
    - Add functionality for viewing, editing, and deleting student records.
    - *Status*: Not Started
- **Attendance Tracking**
    - Create a module for teachers to mark attendance.
    - Generate attendance reports for each student.
    - *Status*: Not Starte

Backlog management will be ongoing throughout the project. During each **Sprint Review** and **Retrospective**, the team will review what was completed, adjust the priorities of the product backlog, and plan for the next sprint.

8. Glossary

- **EMS:** Education Management System
- **CRUD:** Create, Read, Update, Delete
- **UI:** User Interface
- **SRS:** Software Requirement Specifications

9. References

SRS Template:

https://scrumguides.org/docs/scrumguide/v2020/2020-Scrum-Guide-US.pdf